

DAFFODIL INTERNATIONAL UNIVERSITY

Department of Computer Science and Engineering

STUDENT COURSE REGISTRATION SYSTEM

A Data Structure-Based Self-Service Course Registration System

Course Title: Data Structure Lab

Course Code: CSE124

Instructor's Name: Raja Tariqul Hasan Tusher

Submitted by — Group 01

#	Name	Student ID
1	Mst. Mahfuza Anzum Nishat	252-15-388
2	Shahriar Ibn Afjal	261-15-138
3	Md. Nafiz Rayhan	261-15-246
4	Asif Ahmed Rifat	261-15-249
5	Mahdi Hasan	261-15-385

1. Motivation

At our university, students cannot register for courses on their own; each student depends on an assigned academic advisor to complete the registration. This gives students very limited control over their own registration, and any change or correction has to go through the advisor again. It also adds routine, repetitive work for advisors, who must handle many students within a short registration period. A more organized and efficient process is therefore needed. A self-service system can let students view courses, check seats, and register or drop courses by themselves, while the system itself enforces the closed credit system rules so that invalid selections are prevented. This project will improve the registration workflow and, at the same time, demonstrate practical applications of data structures such as hash tables, linked lists, trees, queues, stacks, and graphs.

2. All Objectives

2.1 System-Level Objectives

- Implement a **login system** that verifies the ID and password of a user.
- Create two separate roles, **Student/User** and **Admin**, each with its own menu and permissions.
- Allow students to **view available courses** and their details (code, title, credit, seats, prerequisite).
- Allow students to **register for courses themselves** and to **drop** a registered course where permitted.
- Display a student's **registered courses, total credits, and registration status**.
- Manage **section capacity (seats)** and place students on a waiting list when a section is full.
- Maintain the **closed credit system** by allowing only permitted courses and keeping the total credits within the set limits.
- **Prevent invalid registration**, such as duplicate courses, full sections without waiting, or credit-limit violations.
- Check **prerequisite/eligibility rules** before a course is registered.

- Allow the admin to **add, update, and remove courses**, and to **manage student accounts** and course information.
- Store registration records in files so that data remains available after the program closes.
- Provide a **simple, menu-driven console interface** that is easy to understand and demonstrate.

2.2 Data Structure-Related Objectives

- Apply **appropriate data structures** (hash table, linked list, binary search tree, queue, stack, graph, and arrays) only where each has a clear purpose.
- Demonstrate **insertion, deletion, searching, sorting, and traversal** on real registration data.
- Manage registration records **efficiently**, so that common operations such as login and course lookup are fast.
- Show how the choice of data structure affects the **organization and efficiency** of registration operations.

3. Project Structure

The table below breaks the system into modules and shows the data structure planned for each. All of them can be implemented in C using structures, pointers, and arrays.

Feature/Module	Description	Data Structure	Why It Is Used
Login/User Authentication	Verifies ID and password and opens the Student or Admin menu.	Hash Table (with chaining)	Finds a user by ID in almost constant time; only one record needs to be checked at login.
Student/User Management	Stores student ID, name, department, semester, password, and credit limit.	Hash Table (chained linked lists)	Fast lookup, insertion, and deletion by student ID; collisions are handled by linked lists.
Admin Management	Stores the few admin accounts and their access rights.	Array of structures	The number of admins is small and fixed, so a simple array is enough.

Course Management	Stores course code, title, credit, semester, seats, and prerequisite.	Binary Search Tree (key: course code)	Supports insertion, deletion, searching, and sorted (in-order) listing of courses.
Course Search	Searches a course by code or by title keyword.	BST search; linear search on list	BST search is faster than scanning all courses; title keyword search uses a traversal.
Course Registration	Adds a course to the student's list after all checks pass.	Singly Linked List (per student)	The number of registered courses changes often, so nodes can be added and removed easily.
Course Dropping	Removes a registered course and frees the seat.	Singly Linked List (deletion)	A course can be deleted from anywhere in the list without shifting elements.
Registered Course List	Shows a student's current courses and total credits.	Singly Linked List (traversal)	One traversal displays each course and adds up the credits.
Seat/Capacity Management	Tracks total and remaining seats of each course.	Fields in the course node (BST)	Counters are kept with the course record and updated on every registration or drop.
Waiting List	Keeps students waiting for a full course in order of request.	Queue (linked, one per course)	FIFO order is fair: the first waiting student gets the first free seat.
Prerequisite/Course Dependency	Checks if a student has completed the required earlier courses.	Graph (adjacency list, directed)	Each edge shows "course A is required for course B"; edges are checked before registration.
Registration History	Records recent register/drop actions of a student.	Stack	LIFO order shows the latest action first and can be used to undo the last action.
Student Records	Keeps completed courses and results of each student.	Linked List (per student)	Grows as courses are completed; used by the prerequisite check.
Searching and Sorting	Lists courses by code, credit, or available seats; lists students by ID.	Array (copy of records) with Merge/Bubble Sort	Data is copied into an array and sorted for display without changing the stored structure.

Design note: A graph is used only for prerequisites, where relations between courses naturally form a directed graph. A heap is not included because no feature needs priority ordering; the waiting list uses a simple queue.

4. Output

A. User/Student Features

- Log in and log out with student ID and password.
- View personal information (name, ID, department, semester).
- View the list of available courses for the current semester.
- Search courses by course code or title.
- View course details (credit, prerequisite, seats).
- Check the number of available seats in a section.
- Register for a course after the system checks eligibility, prerequisites, seats, and credit limits.
- Drop a registered course while registration is open.
- View registered courses and **total registered credits**.
- Check registration status (registered, waiting, or dropped).
- View prerequisite/eligibility information of a course.
- View registration history (recent registration and drop actions).

B. Admin Features

- Admin login and logout.
- Add, update, and remove student records.
- Add, update, and remove courses.
- Manage course capacity (seat count).
- Open and close the registration period.
- Manage prerequisites and credit limits used for eligibility checking.
- Search student and course information.
- View the registration records of any student.
- Monitor registered students and the waiting list of each course.
- View simple system statistics (total students, total courses, seats filled per course).

5. Comparative Analysis

The following table compares the current process with the proposed system. The existing-system column is based on how the process currently works from a student's point of view.

Aspect	Existing System	Proposed System
Registration method	Registration is done by the advisor for the student.	The student registers through the system after logging in.
Student involvement	Limited; the student depends on the advisor.	Direct; the student selects, registers, and drops courses.
Advisor workload	The advisor handles registration for many students.	Routine registration work is reduced because the student does it.
Course selection	Chosen with the advisor's help.	Chosen by the student from the courses shown by the system.
Registration accessibility	Depends on the advisor's availability.	Available to the student whenever registration is open.
Seat availability checking	Checked while the advisor processes the registration.	Seats are displayed and updated automatically on each registration or drop.
Registration speed	Depends on advisor and student meeting.	Faster for routine cases, since checks are done by the program.
Record management	Managed by the advisor and university staff.	Stored in files and loaded into data structures by the program.
Error prevention	Relies on the advisor to notice mistakes.	The system checks duplicates, prerequisites, seats, and credit limits.
Course search	Not directly available to the student.	Students can search by course code or title.
Registration monitoring	Monitored through the advisor.	Admin can view registration records and the waiting list.
Administrative control	Handled by the university and advisors.	Admin manages students, courses, seats, and registration status.
Closed credit system	Followed by the advisor.	Kept unchanged; the system enforces the credit rules automatically.
Data management	Manual or separate records.	Organized using hash tables, trees, lists, and queues.

Overall workflow	Student → Advisor → Registration.	Student → Login → Checked registration; Admin manages the data.
-------------------------	-----------------------------------	---

Limitation: The proposed system is a console-based prototype for a lab project. It does not connect to the university's official portal and covers only the basic rules of the closed credit system.